COMP2511

Tutorial 5

Last Week's Tutorial

- Design Principles (SOLID, Law of Demeter)
- Design by Contract
- Streams

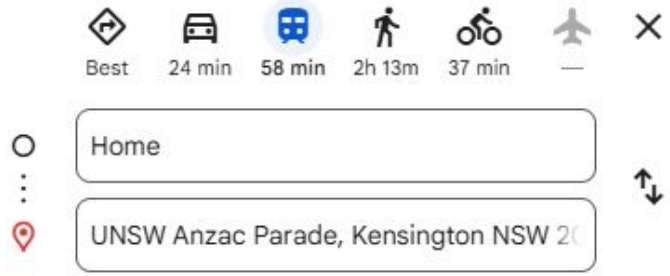
This Week's Tutorial

- Strategy Pattern
- Observer Pattern
- Composite Pattern
- Factory Pattern

Design Patterns

- Design Patterns are typical repeatable solutions to common problems in software design.
  - They are basically the ‘tried and true’ solutions to specific problems.
  - The aim is that they reduce the risk of introducing design flaws when used properly.
- They are not immediate solutions themselves - they represent **templates** that you need to know how to adapt according to the scenario.
  - Design patterns are useful, but don’t use them for the sake of using them.
- **Refactoring Guru**: a great resource that has explanations and examples for each of the design patterns covered in the course.

Strategy Pattern

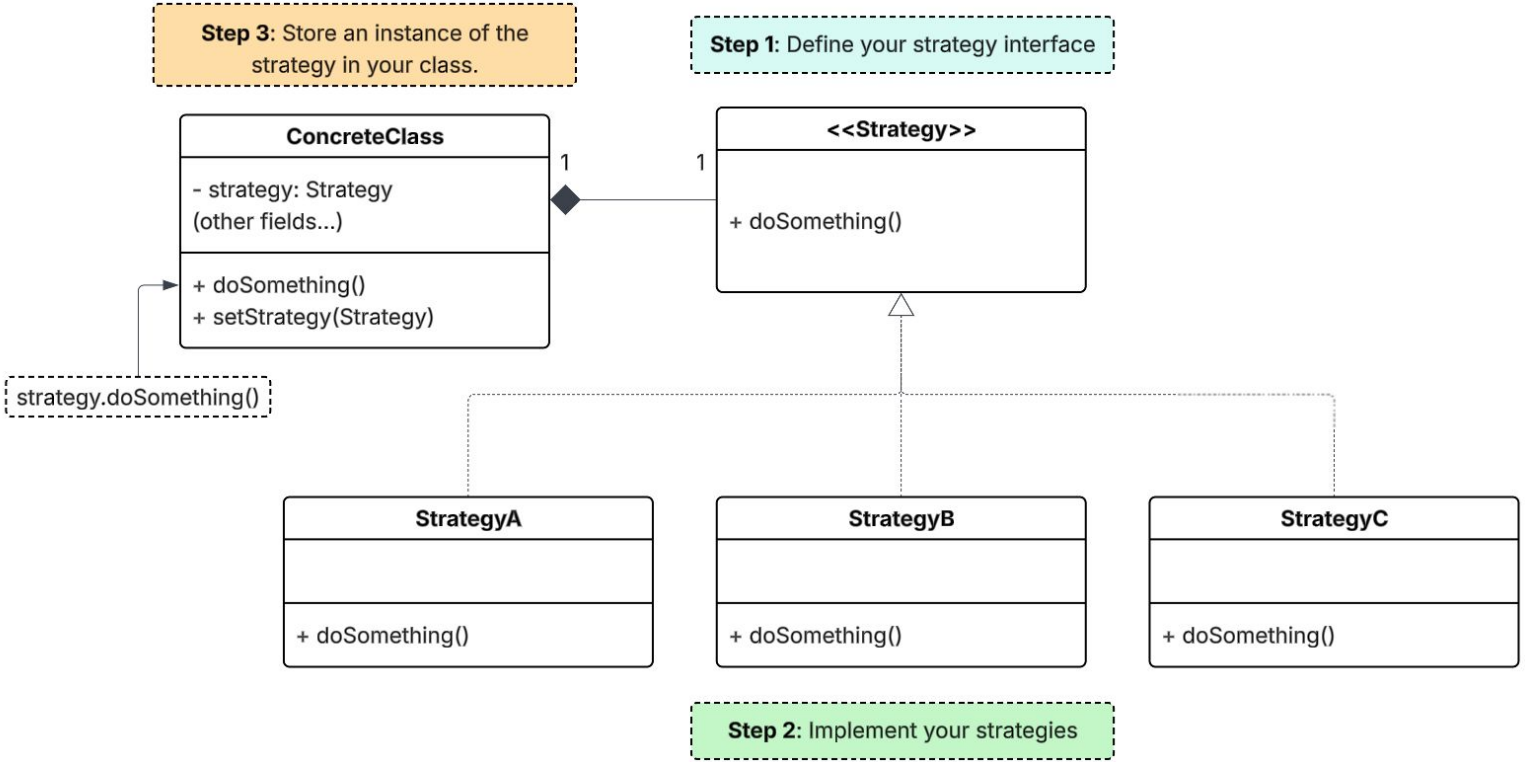


- Consider how you can alternate between different modes of transport while planning a route on Google Maps.
  - The specific details of the route, e.g. the distance you have to cover, the turns you have to take, any costs incurred etc. may be very different across the modes of transport.
  - Regardless, the common aspect of each of these routes is that they all go the same destination.
  - You can think of the transport options as being “different ways to do the same thing”, and you can easily switch between them. This is the main idea of the **Strategy Pattern**.

Strategy Pattern

- The Strategy Pattern is a design pattern that allows a class to change its behaviour at run-time.
- This is relevant when there is a task that can be performed in several different ways (e.g. movement in a game, navigation, sorting order).
- Rather than placing all of the different implementations in a single class, each implementation is separated into different classes called **strategies**.
- Strategies then get stored as a class field. Due to polymorphism, the behaviour of the object will depend on the specific strategy being stored.

Strategy Pattern: Implementation



Live Example: Strategy Pattern

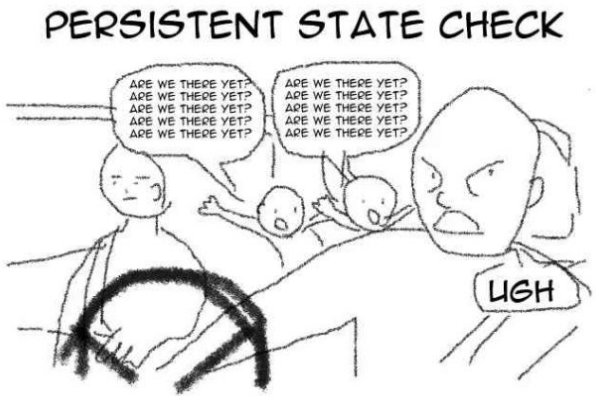
- Inside **src/restaurant** is a restaurant payment system with the following requirements:
  - The restaurant has a menu (a map of menu items to a price).
  - Users can enter their order as a series of meals and the system returns their cost.
- The prices on meals vary in different circumstances. The restaurant has three different price settings:
  - **Standard** - normal rates
  - **Holiday** - 15% surcharge on all items
  - **Member's Discount** - customers with a member's card get a 20% discount
- Refactor the code using the Strategy Pattern. Then, implement a new **Lucky Draw** Strategy - every customer has a 50% chance to receive a 30% discount.

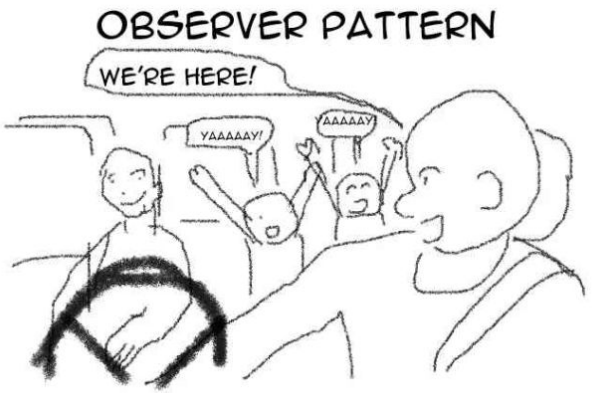
Observer Pattern

- If you were subscribed to a YouTube channel and wanted to watch that channel's videos as soon as they came out, would you rather:
  - Keep refreshing the YouTube channel every 5 seconds?
  - Receive a notification whenever a new video has been uploaded?
- The latter case is more sensible - when a channel publishes a new video, its subscribers get notified.
- We can frame this scenario in a specific way: There is an **event** (a new video being uploaded) that some people are waiting on. As soon as that event happens, everyone waiting gets **notified** and respond to it. This is the main idea of the **Observer Pattern**.

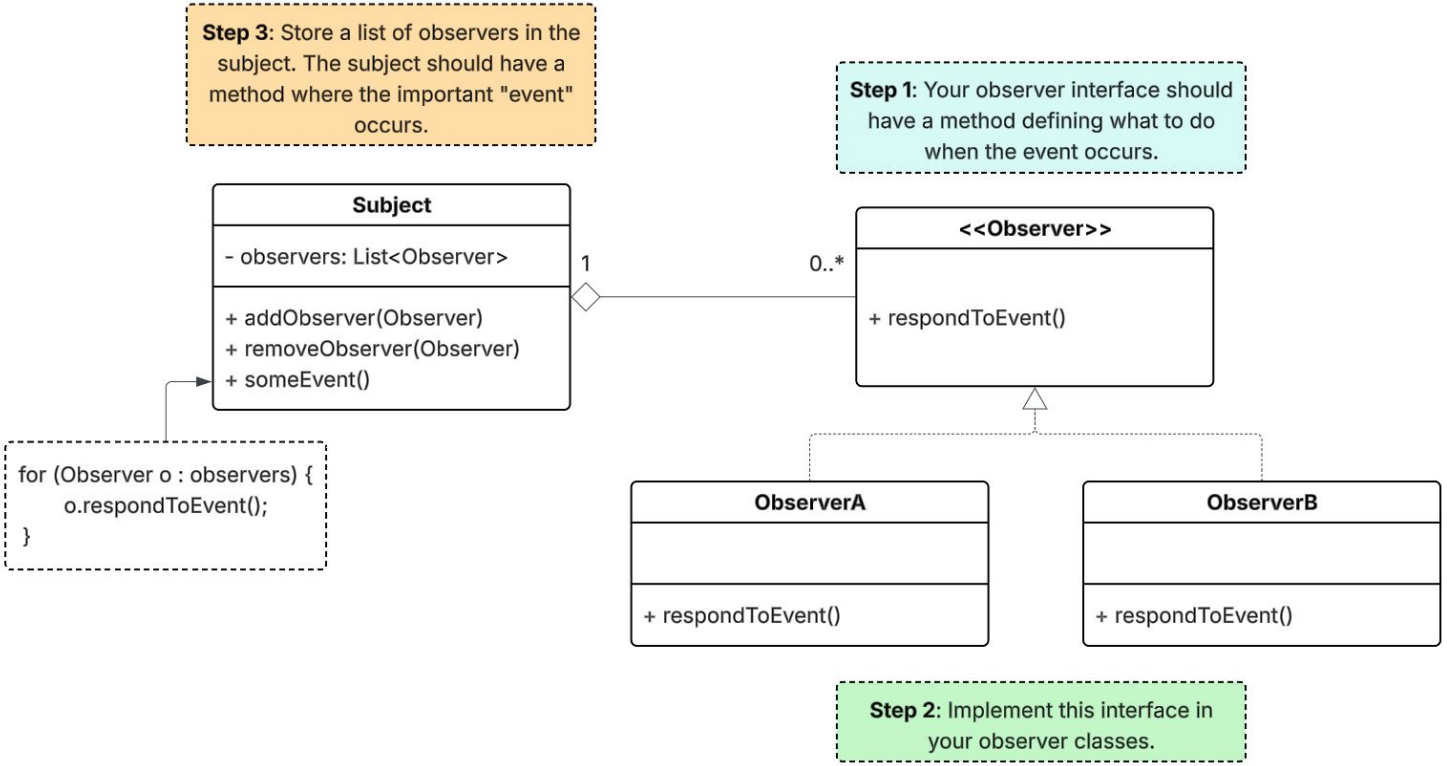
Observer Pattern

- The Observer Pattern is a design pattern where a subject maintains a list of observers, who are notified once an important event happens with the subject.





Observer Pattern: Implementation



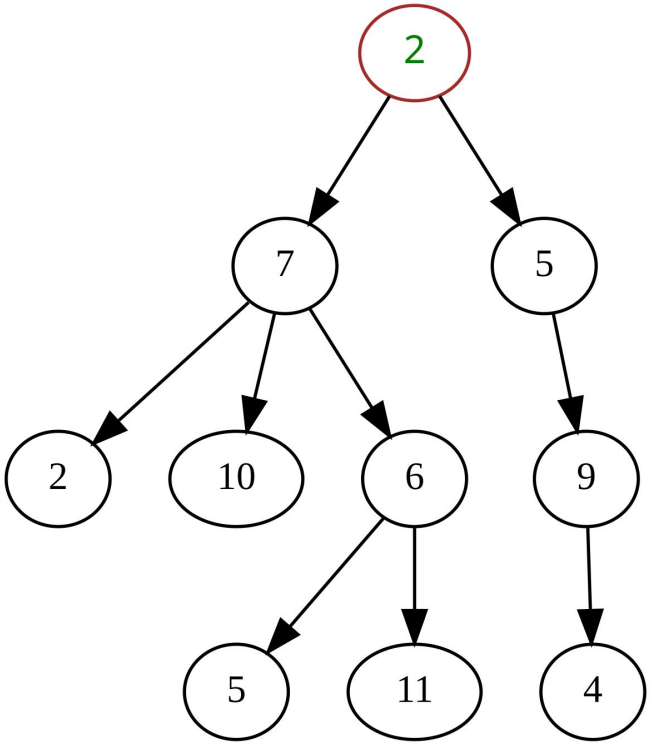
Live Example: Observer Pattern

- In **src/youtube**, create a model for the following requirements of a simple YouTube-like platform using the Observer Pattern.
  - A YouTuber has a name, subscribers and videos.
  - A User has a name and can subscribe to any YouTuber.
  - When a YouTuber posts a new video, each user subscribed to that YouTuber should be notified.
  - Write a simple test with print statements inside YouTube.java.

Composite Pattern

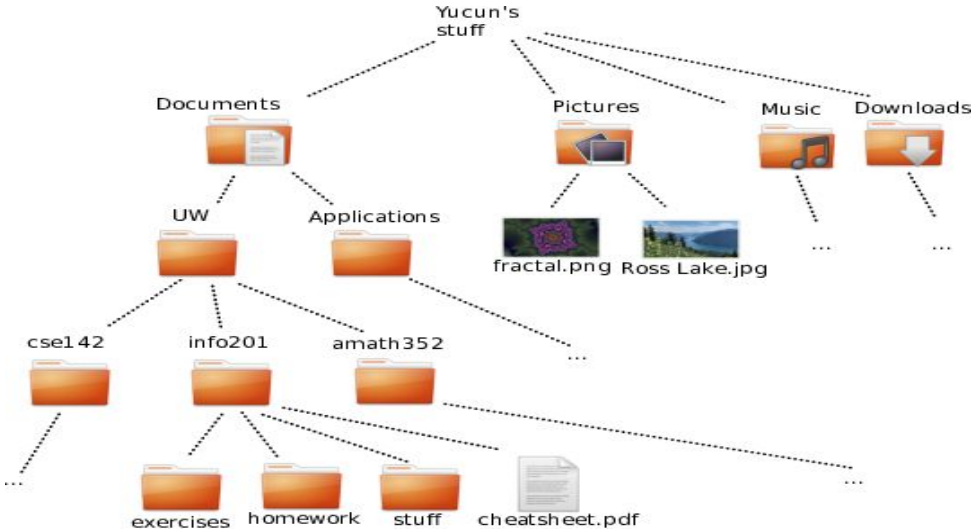
- The Composite pattern is a design pattern that organises objects as a **tree**, i.e. a recursive structure where each object has zero or more subtrees.
  - Nodes that have (non-empty) subtrees as children are referred to as **composites**.
  - Nodes that have no children are referred to as **leaves**.
- The aim of the Composite Pattern is to interact with composite and leaf objects in the same manner. This means that composites and leaves should implement a common interface.
  - Composites need to implement the **recursive case** (uses results of subtrees).
  - Leaves need to implement the **base case** (does not use results of subtrees).

Composite Pattern

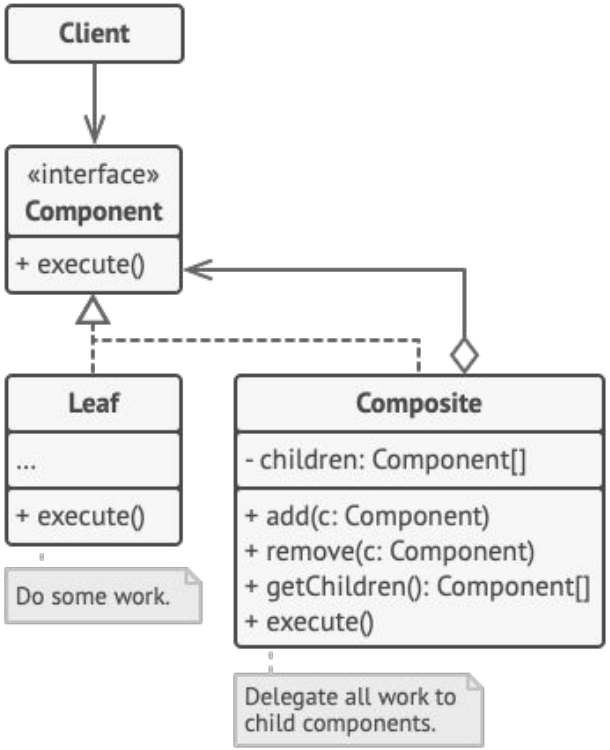


Composite Pattern

- Consider the Linux file system, where everything is a file.
- In this case, directories are composites (can contain other files), while the individual files that contain their own content are the leaves.



Composite Pattern: Implementation

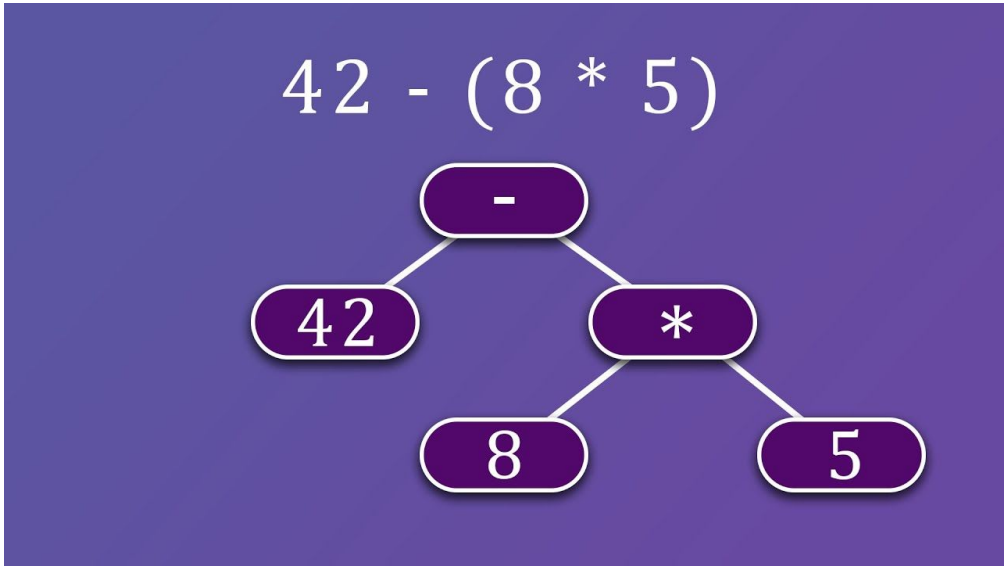


Live Example: Composite Pattern

- An arithmetic expression is a combination of numbers and operators that evaluate to a single value.
- Examples of arithmetic expressions:
  - 6
  - 6 + 7
  - (6 + 7) - 5
  - ((6 + 7) - 5) * 2
- How might this relate to the Composite Pattern? What are the composites and leaves?

Live Example: Composite Pattern

- Inside **src/calculator**, use the Composite Pattern to represent numerical expressions (using +, -, ×, ÷) as a tree.



Factory Pattern

- The Factory Pattern is a design pattern that abstracts away the logic of using and calling constructors to other classes.
- The construction logic is delegated to classes called **factories**. Rather than calling constructors directly, objects get created by calling methods defined in factories, which return objects of the required type.
- This is useful for situations where the logic required to construct an object is complicated and bloats the main program logic.
- **Disclaimer**: You may see the Factory Pattern defined differently in other resources. We will look at the other definition in Week 7.

Live Example: Factory Pattern

- Inside **src/myunsw**, there is a simple system modelling the enrolment of new students into UNSW.
  - To enrol a student, their ID needs to be generated and their program needs to be turned from its string representation to its numeric code (e.g. Computer Science → 3778).
- **Refactor the code to use the Factory Pattern to abstract the Student creation logic away from the MyUNSW class.**